

# INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo • Ingeniería en Sistemas Computacionales

## Ejercicio 1. Control de Versiones con Git y GitHub

**Unidad de Aprendizaje:** Bases de Datos • **Práctica:** 1 – Modelo Entidad Relación

**Alumno:** García Verduzco Fernando • **Entrega:** Septiembre 2026

### 1. Sistema de control de versiones y resolución de conflictos colaborativos

Un Sistema de Control de Versiones (VCS, por sus siglas en inglés) es una herramienta de infraestructura de software concebida para registrar, clasificar y auditar de forma cronológica, determinista e incremental el conjunto de modificaciones aplicadas a archivos de código fuente, scripts de definición de bases de datos y esquemas documentales a lo largo del tiempo.

En el ámbito del trabajo colaborativo en equipo, los VCS resuelven de manera definitiva problemas operacionales críticos:

- **Eliminación de la sobrescritura destructiva:** Sin un VCS, el acceso simultáneo de varios desarrolladores a un mismo recurso provoca que los cambios del último guardado anulen de forma silenciosa el trabajo previo de sus pares. El sistema automatiza o asiste en la convergencia de líneas de edición independientes.
- **Trazabilidad y auditoría granular:** Cada cambio queda catalogado mediante identificadores criptográficos inmutables, estampas de tiempo y firmas de autoría, permitiendo conocer con absoluta certeza el origen, contexto e intencionalidad técnica de cada modificación.
- **Aislamiento y experimentación segura:** La bifurcación en ramas independientes garantiza que las funcionalidades en desarrollo o pruebas de estrés no desestabilicen el entorno central ni el código productivo validado.
- **Recuperación inmediata ante regresiones:** Provee mecanismos formales para revertir de manera determinista el árbol de trabajo a un estado histórico funcional en caso de fallos críticos o incompatibilidades en compilación.

### 2. Diferenciación técnica y conceptual entre Git y GitHub

Existe una distinción categórica y arquitectónica entre Git y GitHub; bajo ningún esquema deben emplearse como términos sinónimos:

**Git** es un motor distribuido de control de versiones ejecutado en el espacio de usuario local. Fue concebido bajo una arquitectura basada en un Grafo Acíclico Dirigido (DAG), donde los objetos internos representan instantáneas completas del sistema de archivos y sus metadatos asociados. Todas las operaciones principales (confirmación, bifurcación, inspección del árbol de diferencias y fusión) se ejecutan de manera local sin dependencia de conectividad de red ni servidores centralizados.

**GitHub**, en contraste, es una plataforma de servicios en la nube orientada a la gobernanza y hospedaje de repositorios remotos que utilizan el protocolo Git. Adiciona una capa empresarial sobre el núcleo del motor: interfaces web de revisión de código, control de acceso basado en roles (RBAC), seguimiento de incidencias (issues), canalizaciones automatizadas de integración

y despliegue continuo (CI/CD mediante GitHub Actions) y flujos formales de evaluación por pares mediante *Pull Requests*.

### 3. Definición formal de términos técnicos

- **Repositorio (Repository):** Estructura integral de base de datos administrada por Git (ubicada en la carpeta oculta `.git`) que almacena el grafo inmutable de confirmaciones, referencias de ramas, etiquetas y el árbol histórico completo del proyecto. *Ejemplo:* La carpeta `practica1-bd/` que resguarda el código fuente, esquemas SQL y documentación del curso.
- **Confirmación (Commit):** Objeto criptográfico inmutable identificado unívocamente por una suma de comprobación SHA-1 que representa una instantánea atómica del árbol de archivos en el área de preparación (stage), acompañada de metadatos de autoría y mensaje descriptivo. *Ejemplo:* `commit a46406b - "Agrega README inicial de la practica"`.
- **Rama (Branch):** Puntero de referencia móvil que apunta a la punta o última confirmación registrada de una línea de desarrollo divergente e independiente. *Ejemplo:* La rama `feature/estado-del-arte` utilizada para redactar las fichas sin intervenir en `master`.
- **Fusión (Merge):** Algoritmo de integración mediante el cual Git combina el historial cronológico y los contenidos de dos o más ramas divergentes en un commit de integración unificado. *Ejemplo:* La incorporación de las modificaciones de `feature/estado-del-arte` sobre la rama base `master`.
- **Conflicto de fusión (Merge Conflict):** Estado en el cual el algoritmo de combinación de Git se detiene debido a que dos confirmaciones concurrentes alteraron las mismas líneas de un archivo de manera discrepante, demandando resolución manual por parte del desarrollador. *Ejemplo:* Dos colaboradores modificando en simultáneo los parámetros de conexión del puerto expuesto en `compose.yaml`.
- **Pull Request (PR):** Solicitud formal implementada en plataformas remotas donde un colaborador propone incorporar el trabajo de una rama secundaria a una rama protegida, habilitando la discusión técnica, la validación de pruebas automáticas y la aprobación colegiada. *Ejemplo:* La apertura del PR #1 en GitHub para validar la integración de la documentación científica.
- **Archivo `.gitignore`:** Archivo declarativo de texto que define patrones de expresiones regulares y nombres de archivo que el motor de Git debe omitir de forma determinista para evitar su inclusión accidental en el seguimiento. *Ejemplo:* Reglas para excluir variables de entorno sensibles (`.env`) o directorios de artefactos generados.
- **Archivo `README.md`:** Documento estándar de bienvenida e inducción técnica escrito en formato Markdown, diseñado para servir como catálogo de navegación del proyecto, requisitos del entorno y autoría académica. *Ejemplo:* El archivo `README.md` en la raíz del repositorio con los datos del alumno y la estructura de navegación.

### 4. Flujos de trabajo basados en ramas y revisión entre pares (Peer Review)

Los flujos de trabajo contemporáneos basados en ramas (como *GitHub Flow*) estipulan que la rama troncal (`main` o `master`) debe mantenerse permanentemente en un estado validado, compilable y listo para su liberación a producción. Por consecuencia técnica, ninguna adición de código, corrección de defectos o integración documental se aplica directamente sobre el tronco; se crean ramas secundarias de ciclo de vida corto orientadas a características específicas.

La revisión formal de código entre pares antes de la fusión representa una salvaguarda esencial por los siguientes fundamentos de ingeniería:

- **Garantía de calidad y mitigación de defectos:** La examinación colegiada permite detectar inconsistencias semánticas, errores de diseño relacional, fallos de seguridad (tales como exposición de credenciales) y violaciones de estándares que comúnmente pasan desapercibidos para el autor original.
- **Distribución del conocimiento técnico:** Asegura que múltiples ingenieros del equipo comprendan los módulos arquitectónicos incorporados, erradicando el riesgo operativo asociado a dependencias exclusivas de un único desarrollador (conocido como factor de riesgo o *bus factor*).
- **Mantenibilidad y uniformidad de la base de código:** Estandariza las pautas de nomenclatura, documentación interna y convenciones de codificación en todo el proyecto.